

Deep Dive: OOXML, Macros & Steganography

The upstream case, solved together — the two new challenges stay yours.

Stefanus Eko Prasetyo

Lab-3 · companion to the Lab 03 lecture

What we cover today

1 OOXML in depth

unzip workflows, the parts that matter, hidden text, external relationships

2 Macros in depth

olevba output anatomy, auto-run hooks, obfuscation families

3 Tool packs

exiftool, zsteg plane grammar, steghide, spectrograms with sox/Audacity

4 Worked examples

the upstream case: the docm macro, the mole's photo metadata, the audio

5 Your turn

Challenge B (the cover letter) and Challenge C (the harbor photo)

Unzip, then read — the full workflow

```
student@dfir:~/lab03$  
  
$ unzip -l Doc1.docx          # list without extracting  
Length  Name  
  1291  word/document.xml  
   590  docProps/core.xml  
  
$ unzip Doc1.docx -d doc1_x/  && cd doc1_x  
$ xmllint --format word/document.xml | less  
<w:t>Hello World!</w:t>  
$ grep -c 'w:t' word/document.xml  
$ grep -o '<w:vanish/>' word/document.xml | wc -l  
# vanish = runs Word does NOT display - pure XML gold
```

Workflow habits

- ▶ list first (-l), extract second, to a named folder
- ▶ xmllint --format makes one-line XML readable
- ▶ grep answers count questions fast
- ▶ compare parts against a clean file you create yourself

Relationships — including the ones that phone home

```
student@dfir:~/lab03$  
$ cat word/_rels/document.xml.rels  
<Relationship Id="rId1" Type=".../styles"  
  Target="styles.xml"/>          # internal part  
<Relationship Id="rId9"  
  Type=".../attachedTemplate"  
  Target="https://templates.corp-tools-example.com/  
    normal.dotm"  
  TargetMode="External"/>      # EXTERNAL!  
  
# remote template = code fetched at open time (Moniker)
```

What to check

- ▶ every rels file, at every level
- ▶ TargetMode="External" is the magic string
- ▶ oleid counts these for you: 'External Relationships'
- ▶ ask: does a document really need to reach the internet?

What hides inside the parts

Technique	Where it lives	How you find it
Hidden text	<code><w:vanish/></code> on runs in document.xml	grep vanish - screen shows nothing, XML shows everything
White-on-white text	color FFFFFFFF runs	read the XML, not the render
Tiny text	sz 2 runs between normal lines	formatted dumps reveal size jumps
Off-slide content	shapes positioned far outside the slide	read slide XML coordinates
Author & timestamps	docProps/core.xml	cat core.xml - creator, created, modified
Editing time & app	docProps/app.xml	TotalTime, Application, Template

Reading olevba output like a pro

```
student@dfir:~/lab03$  
$ olevba YearlyBonus.docm  
-----  
VBA MACRO ThisDocument.cls  
in file: word/vbaProject.bin - OLE stream: 'VBA/ThisDocument'  
-----  
Sub AutoOpen()  
    Dim payload As Variant ...  
-----  
ANALYSIS:  
+-----+-----+-----+  
| Type      | Keyword    | Description                               |  
| AutoExec  | AutoOpen   | runs when the document is opened        |  
| Suspicious | ByteArray  | may decode/execute embedded payload     |
```

Anatomy

- ▶ one block per VBA module, with its OLE stream name
- ▶ ANALYSIS table flags AutoExec, Suspicious, IOC keywords
- ▶ Hex/Base64 strings are auto-decoded below the table
- ▶ triage rule: AutoExec + ByteArray + Shell = deep dive

Obfuscation families you will meet

Family	Looks like	Unmask it with
Encoded strings	<code>Chr(65)&Chr(66)...</code> / Base64 / Hex blobs	olevba auto-decode; CyberChef
Byte arrays	<code>raw = Array(94, 111, 119...) then decode</code>	small shift/XOR loop in Python
Dead code	hundreds of unused Dim lines	read the Auto* subs only
String stacking	<code>payload = a & b & c</code> built piecewise	concatenate by hand or script
VBA stomping	compiled p-code differs from source	olevba warns; use pcodedmp

The universal move

- however scary the obfuscation, the macro must eventually produce a string the machine understands - find the decode loop and re-run it yourself in Python, offline

exiftool options worth knowing

Command	What it does
<code>exiftool img.jpg</code>	full metadata dump
<code>exiftool -a -u img.jpg</code>	ALL tags, including unknown/duplicated ones
<code>exiftool -gps:all -a img.jpg</code>	GPS block only, all instances
<code>exiftool -Artist -Software img.jpg</code>	just the named fields
<code>exiftool -csv -r cases/ > meta.csv</code>	batch a whole folder to one CSV
<code>exiftool docm/pptx files</code>	Office props too: Author, Company, dates

Remember: GPS 34 deg 12' 39" N = 34 + 12/60 + 39/3600 = 34.2109 - decimal degrees drop straight into any map.

zsteg — reading the plane grammar

```
student@dfir:~/lab03$  
  
b1 , rgb , lsb , xy      .. text: "..."  
| | | |  
| | | +-- scan order: xy=row-major, YX=column-major  
| | +----- bit order: lsb or msb  
| +----- channels: rgb, rgba, r, g, b  
+----- bit plane: b1 = LSB ... b8 = MSB  
  
$ zsteg -a harbor.png      # try ALL combinations  
$ zsteg -E b1_rgb_lsb_xy harbor.png # export the plane
```

Pro habits

- ▶ read the FIRST hit - usually b1,rgb,lsb,xy
- ▶ file: hits (PNG/OpenPGP) mean structured data in that plane
- ▶ compare with a re-saved image: real LSB stego survives
- ▶ install: `gem install zsteg` (needs Ruby)

steghide & spectrogram tooling

```
student@dfir:~/lab03$  
$ steghide info cover.jpg      # does it carry data?  
$ steghide extract -sf cover.jpg -p "" # empty passphrase  
wrote extracted data to "secret.txt".  
$ steghide extract -sf cover.jpg -xf out.bin  
  
# spectrograms without the GUI:  
$ sox audio.wav -n spectrogram -o spec.png  
$ sox audio.wav -n spectrogram -x 1600 -o spec.png # wider
```

Notes

- ▶ steghide also embeds in BMP and WAV
- ▶ passphrases: try empty, then hints from metadata
- ▶ Audacity: track menu -> Spectrogram; zoom to read
- ▶ brighter pixels = more energy at that frequency/time

YearlyBonus.docm — the macro's secret

student@dfir:~/lab03\$

```
$ olevba YearlyBonus.docm | less
Sub AutoOpen()
  Dim raw As Variant
  raw = Array(94, 111, 119, 44, 32, 121, 111, 117, ...)
  ' decode and show the secret ...

# re-run the loop ourselves (shift +10):
$ python3 -c "print(bytes((b+10)&0xFF for b in raw))"
Wow, you really know your way around a VBA
macro challenge. Your flag is below:
```

Upstream flag (original format):

flag{such_m4cr0_much_w0w!}

Method

- ▶ olevba gives the array
- ▶ you owe the +10 shift
- ▶ never execute the macro to find out

Presentation.pptx — unmasking the mole

```
student@dfir:~/lab03$  
$ unzip Presentation.pptx -d ppt_x/ && cat ppt_x/ppt/slides/_rels/slide1.xml.rels  
  first embed: rId2 -> ../media/image1.jpg      # the smuggled photo  
$ exiftool -Artist -GPS:all -a ppt_x/ppt/media/image1.jpg  
  Artist       : Michael Scott          <- name of the source  
  GPS Position : 34 deg 12' 39.00" N, 118 deg 26' 11.00" W  
                = 34.2109, -118.4364 -> Burbank, Los Angeles, California  
  
$ steghide extract -sf image1.jpg  # empty passphrase, on Kali  
  -> the top secret message carried inside the JPEG  
# name + location + payload = the mole's full profile
```

super_secret_audio.wav — the spectrogram flag

student@dfir:~/lab03\$

```
# Audacity: open -> track menu -> Spectrogram
# or render headless:
$ sox super_secret_audio.wav -n spectrogram -o spec.png
$ eog spec.png

# the bright shapes across the mid frequencies spell:
flag{h1dd3n_1n_th3_n01s3}

# to the ear it is just noise; to the eye it is a banner
```

Upstream flag (original format):

flag{h1dd3n_1n_th3_n01s3}

Report it

- ▶ your working paper needs the screenshot
- ▶ note the tool and the view used

Common mistakes, proven habits

Mistake	Habit instead
Opening the original file in Office	unzip a copy; read the XML - zero execution risk
Clicking 'Enable Content' to 'see the macro'	olevba reads the code; you never need to run it
Trusting metadata as proof	metadata is evidence, not verdict - it can be spoofed; corroborate
Stopping at the first zsteg line	scan the whole output; false planes are normal noise
Forgetting the audio channel	when images are clean, listen - and look at the spectrogram

Two fresh artifacts. Same discipline.

01

Challenge B — 4.2

cover_letter.docx: a job application that looks perfectly innocent on screen. Take it apart as a ZIP - the XML, the properties, and the relationships each tell a different story.

02

Challenge C — 4.3

harbor_photo.png: a stock photo whose metadata says 'nothing here'. Run the plane grammar over it and recover the smuggling note and the flag.

03

Working paper

pre-lab first; staged hints in section 4; evidence numbering B-xx / C-xx; hashes before and after.